The Connection Cubes : Symmetric, Low Diameter Interconnection Networks with Low Node Degree

Nitin K. Singhvi

Department of Computer Science
State University of New York at Binghamton, NY 13902–6000.

Abstract

The Enhanced Connection Cube or ECC and the Minimal Connection Cube or MCC, proposed in this paper, are regular and symmetric static interconnection networks for large–scale, loosely coupled systems. The ECC connects 2^{2n+1} processing nodes with only n+2 links per node, almost half the number used in a comparable hypercube. Yet its diameter is only n+2, almost half that of the hypercube. The MCC connects 2^{2n+1} nodes using only n+1 links per node, has about the same diameter as a hypercube and is scalable like the hypercube. The MCC can be converted into the ECC by adding one more link per node. Both networks can emulate all the connections present in a hypercube of the same size, with no increase in routing complexity, so that typical parallel applications run on both types of CCs with the same time complexity as on a hypercube.

Keywords: hypercube, interconnection networks, massively parallel, parallel processing.

1: Introduction

Static interconnection networks in which point–to–point links connect pairs of nodes appear to be the networks of choice for large scale, parallel processing systems or massively parallel systems. Direct communication paths are available between nodes and locality of communication can be efficiently supported. One of the most popular point–to–point topologies is the binary n–cube or hypercube network [12], in which n links are incident on a node for a network containing 2^n nodes. A variety of hypercube–based interconnection networks have been proposed [3–10] and [14].

The hypercube is scalable and symmetric and has a logarithmic diameter, simple routing and broadcasting algorithms and subsumes commonly required interconnection patterns such as meshes, trees and rings. However it does not attain the lower diameter and lower average internode distance that is possible with the number of links per node that it possesses. It has been observed that the hypercube interconnection does not use all the links effectively [11] since a large number of optimal routes exist between most pairs of nodes. Alternatively, it can be viewed as a structure that uses "too many" links to achieve its advantages. As mentioned in [12], there appear to be two approaches taken so far in designing cube–based interconnections to increase the efficiency of link usage. In one approach, the number of links is kept the same but the diameter is reduced to about half that of a comparable hypercube along with a reduction in the average internode distance, eg. the twisted Cube [7], the Mobius cubes [4] and the Mcube [12]. In the other approach, the number of links is reduced, with a preservation of diameter, or even with a reduction in diameter, eg. the HCN [6]. Lower diameter is desirable because it lowers the expected communication delay, thereby enhancing system performance. A lower diameter also lowers the time required for broadcasts, which are often essential in parallel applications. However, asymmetry in the twisted Cube [7] has been shown in [1] to have an adverse effect on network traffic. In fact, it has been shown in [1] that asymmetry causes premature saturation of certain links and large message queues at certain nodes under heavy message traffic, resulting in an increase in message delays. The Mcube [12] does preserve symmetry while achieving the advantages of [4] and [7] but follows the first approach and requires the same amount of links as a hypercube.

The Connection Cubes, proposed in this paper, are concerned with the second approach, in which the number of links are reduced in comparison with a hypercube. The HCN [6] follows this approach and, in fact, does achieve a lower diameter even with a reduction in the number of links. However, it is not a symmetric structure. Further, it is not easily scalable without the removal of certain existing links. As mentioned in [6], reducing the number of links has useful implications in the VLSI implementation of a node in a large network. The Connection Cubes provide the advantages of reduction in the number of links to about half of that in a

1063-7133/93 $3.00 © 1993 IEEE

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

hypercube, and can emulate all the connections of a hypercube, for parallel algorithms, with no increase in routing complexity. The Enhanced Connection Cube or the ECC connects 2^{2n+1} nodes with only n+2 links incident on each node and yet has a diameter of n+2. A hypercube of 2^{2n+1} nodes has 2n+1 links incident on each node and a diameter of 2n+1. So an ECC uses almost half the number of links used by a similar sized hypercube and yet has almost *half* the diameter of the hypercube. The Minimal Connection Cube or MCC connects 2^{2n+1} nodes with only n+1 links incident on a node. It has about the same diameter as a hypercube and is scalable without any alteration to existing links. The MCC can, with the addition of a single link to each node, be converted into the ECC. Both are symmetric networks in which all nodes have the same distance distribution functions (defined below). This is a desirable characteristic in terms of balancing message traffic. The ECC has about the same average internode distance as a comparable hypercube and the MCC has one slightly higher. The MCC and the ECC thus appear to be viable interconnection networks for large scale, distributed memory systems.

1.1: Preliminary definitions

1) Point–to–point links (or links), in a network, are point–to–point connections between pairs of nodes in the network.
2) The length of a path from a node x to a node y in a network G, is the number of links in the network, crossed by that path.
3) The distance d(x,y), of a node y from a node x, in a network G, with respect to a routing algorithm R, is the length of the shortest path from x to y in G, using R.
4) The diameter of a network, with respect to a routing algorithm R, is max{d(x,y)} over all possible pairs (x,y) of nodes in G, using R.
5) Let G be a network of N nodes with diameter d. Then the distance distribution function for a node j, in G, with respect to some chosen routing algorithm R, is $D_j(i)$ which gives the number of nodes in G, at a distance i from j, using R. We say that G has a uniform distance distribution if and only if $D_j(i)$ is the same for all nodes j in G. The average path delay (APD(j)) for a node j in G and the average internode distance in G (AD(G)) are, respectively,

$$\frac{1}{N}\left(\sum_{i=1}^{i=d}(i * D_j(i))\right) \quad \text{and} \quad \frac{1}{N}\left(\sum_{j=1}^{j=N} APD(j)\right)$$

1.2: Notation list

1) For a binary number x, we use $\bar{x}$ to denote the bitwise complement of x.
2) We use $x \rightarrow m \rightarrow y$ to denote a path of length m from a node x to a node y, in a network.
We use $x \rightarrow m \rightarrow y \rightarrow r \rightarrow z \rightarrow \ldots$ to denote the concatenation of $x \rightarrow m \rightarrow y$, $y \rightarrow r \rightarrow z, \ldots$
3) We use **d(x,y)** to denote the distance from a node x to a node y in a network, with respect to a chosen routing algorithm.

1.3: Construction of a hypercube

In the static hypercube network of dimension n (also called an n–cube), 2^n nodes are given distinct binary addresses using n bits, and two nodes are connected by a link if and only if their binary addresses differ in exactly one bit position. The link connecting two nodes is numbered j (denoted link#j) if the position of the differing bit in their binary addresses is j. Bit positions are numbered 0 to n–1.

2: The Minimal Connection Cube or MCC

The MCC is constructed by taking two sets of unconnected hypercubes and then connecting the two sets by adding just one more link to each node.
An MCC of dimension 2n+1 contains 2^{2n+1} nodes and is constructed as follows:
Let S_0 *and* S_1 *be two sets each containing* 2^n *hypercubes of dimension* n *(n–cubes).*
Let $C_0, C_1, \ldots, C_{N-1}$ *be the n–cubes in each set, where* $N = 2^n$.
Let (k,j,i) *denote the node with binary address* i, *in cube* C_j, *in set* S_k, *then*
Connect node (0,j,i) to node (1,i,j) $\forall$ **i, j** $\ni 0 \le i,j < 2^n$ **—(I)**
(Note that (0,i,i) is connected to (1,i,i), for $0 \le i < 2^n$).

The connection specified in (I) is called the **Dimension Interchange (or DI) Connection.** We refer to links within each n–cube as **internal links** and to links specified in (I) as **interset links.** Therefore each node in an MCC of dimension 2n+1, has n internal links (link#0 to link#n–1) and 1 interset link, giving it a node degree of n+1. Using n bits to represent each of i and j and one bit to represent k one can concatenate k, j and i to form a binary address of the node (k,j,i).

3: Routing and diameter in the MCC

In this Section we present a fully distributed routing algorithm and show that an MCC of dimension 2n+1, with 2^{2n+1} nodes, has a diameter of 2n+2.

The two cases we consider are, (i) where the source and the destination are in different sets and (ii) where both are in the same set. The source node is **(k,j,i)** and the destination node is **(r,q,p)** where j, i, q and p are represented by n bits

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

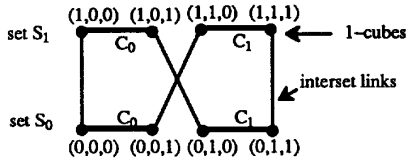

Fig 1. MCC of dimension 3

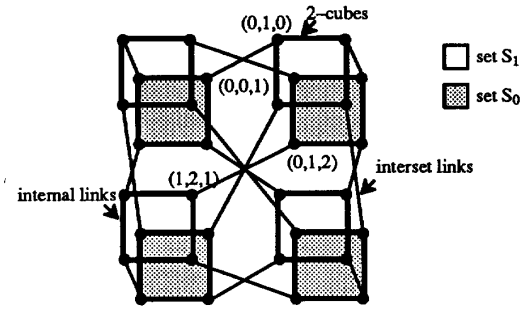

Fig 2. MCC of dimension 5

each, and k and r are represented by one bit each. Note that each set S_0 and S_1, consists of n–cubes, each of diameter n.

Case (i) : r ≠ k.

Route1 : $(k,j,i) \rightarrow x_1 \rightarrow (k,j,q) \rightarrow 1 \rightarrow (r,q,j) \rightarrow y_1 \rightarrow (r,q,p)$ $\{ x_1, y_1 \leq n \}$

Case (ii) : r = k.

If q = j then use **Route2 :** $(k,j,i) \rightarrow x_2 \rightarrow (k,j,p)$ $\{ x_2 \leq n \}$

else use **Route3 :** $(k,j,i) \rightarrow 1 \rightarrow (\bar{k},i,j) \rightarrow x_3 \rightarrow (\bar{k},i,q) \rightarrow 1 \rightarrow (k,q,i) \rightarrow y_3 \rightarrow (k,q,p) = (r,q,p)$ $\{ x_3, y_3 \leq n \}$

Route1, Route2 and Route3 have maximum lengths of 2n+1, n and 2n+2 respectively. ∴ ***The diameter of an MCC of 2^{2n+1} nodes is 2n+2 under the above routing algorithm, equaling the diameter of a hypercube of 2^{2n+1} nodes + 1.***

The following algorithm, for the above routing, can be implemented in a fully distributed fashion:

```
if r = k̄ then
   if i = q then
        send message over the interset link;
   else use hypercube routing to send message over internal
        links to (k,j,q);
else /* if r = k */
   if j = q then
        use hypercube routing to send message over internal
        links to (r,q,p);
   else send message over interset link;
```

4: Uniform distance distribution in the MCC

Note that d((k,j,i), (k,j,p)) = d(i,p) in cube C_j in set S_k which is the distance between i and q using a hypercube routing algorithm. Using this we can express the distance distribution function of a node x where x = (k,j,i), in a (2n+1)–dimensional MCC, with respect to the routing algorithm described above :

$D_x(s) = |A_1| + |A_2| + |A_3|$ where each A_i is a subset of nodes in the MCC, defined as follows:

$A_1 = \{(\bar{k},q,p) \mid d(i,q) + d(j,p) + 1 = s\}$

$A_2 = \{(k,j,p) \mid d(i,p) = s\}$ and

$A_3 = \{(k,q,p) \mid d(i,p) + d(j,q) + 2 = s\}$.

Since a hypercube is known to have a uniform distance distribution under standard routing algorithms, it follows that $D_x(s)$ remains the same for any allowable values of k, i and j, for any given s, i.e. distance distribution is uniform in the MCC. Therefore the Average Internode Distance (AD) equals the Average Path Delay (APD) of any chosen node. In an MCC G, of dimension 2n+1, $AD(G) = (2n+3)/2 + 1/2^n$ (Proof is omitted due to space constraints).

$\therefore AD(G) = (2n+1)/2 + 1 - 1/2^n = AD(H) + 1 - 1/2^n$ where H is a hypercube of dimension 2n+1.

5: Broadcasting

A broadcast in an MCC of dimension 2n+1 can be conveniently achieved in 2n+2 steps with the use of a "counter" in the header of the broadcasted message as follows:

if message is received along internal link with counter = c then

(i) continue with standard hypercube broadcast within the n–cube, keeping counter = c and

(ii) if $c > 0$ then send message along interset link with counter = $c - 1$;

else /* if message is received along an interset link */
if $c > 0$ then follow standard hypercube broadcast within the n–cube, keeping counter = c;

The source node initiates the broadcast by sending the message on all links with the counter = 2.

6: Hypercube emulation

The MCC (and ECC) can emulate a hypercube of the same size as itself, in the execution of algorithms by providing the same kind of conflict free routing as provided in the hypercube. The emulation of the hypercube can be done in different ways depending on the requirements of the algorithm. A hypercube of dimension 2n+1 has links such that any node with a 2n+1 bit address 'x' is connected to all nodes with address 'y' where y differs from x in exactly one bit. As explained earlier, let the address of a node (k,j,i), in a (2n+1)–dimensional MCC be a concatenation of the binary values of k, j and i where j and i are represented by n bits each, and k is represented by 1 bit. Hence in the binary address of a node the 0th thru the $(n-1)^{th}$ bits represent i, the

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

n^{th} thru the $(2n-1)^{th}$ bits represent j and the $2n^{th}$ bit represents k. Each node (k,j,i) in the MCC is directly connected with nodes (k,j,m) where the binary value of m differs from that of i in exactly one bit position. Hence each node is directly connected to each of its peers in the lower n dimensions. Now consider nodes (k,j,i) and (k,q,i) where the binary values of j and q differ in exactly one bit. Such nodes, though connected in a hypercube, are not connected in the MCC. However, if the data elements in each of these two nodes are routed over the interset links they reach nodes $(\bar{k},i,j)$ and $(\bar{k},i,q)$, respectively, in a single step. Since the binary values of j and q differ in exactly one bit, $(\bar{k},i,j)$ and $(\bar{k},i,q)$ are connected. Hence by exchanging all elements between the two sets S_0 and S_1 all data elements are reconfigured so that each element is now directly connected to its peers in the upper n dimensions.

Type–1 emulation: In the address of each node (1,j,i) in set S_1, we *logically* interchange the n bits of j with the n bits of i to get (1,i,j). As a result each node in set S_1 is now connected to all other nodes whose address differs from its own in one of the nth thru the $(2n-1)^{th}$ bits. Set S_0 can now serve to provide a "hypercube connection" for all data, for dimensions 0 to n–1 and set S_1 can do the same for dimensions n to 2n–1. The emulation can now proceed as follows :

1) Execute operations over dimensions 0 to n–1 for all data elements in S_0 and over dimensions n to 2n–1 for all data elements in S_1.
2) Exchange data elements between nodes (0,i,j) and (1,j,i) $\forall$ i, j, over the interset link. (Exchange data elements between the two sets).
3) Execute operations as in (1).
4) The interset links themselves serve to connect data elements over the uppermost dimension, 2n (since (0,j,i) is connected to *logical* (1,j,i)).

Therefore, for any pair of data elements that are connected over some dimension r in the hypercube, with $0 \le r < 2n+1$, the following is true: The corresponding data elements, in the MCC, are either connected by an existing link or require just one routing step to be so connected.

Type–2 Emulation: In the alternative, we do not logically alter any addresses, and both sets can serve to connect data elements in the lower n dimensions in step (1). Step (2) can then be used to exchange elements between sets S_0 and S_1 and both sets can serve to connect data elements in the upper n dimensions in step (3). This may be more convenient for certain algorithms. This, however, prevents the interset links from being used as connections over the uppermost dimension. A data element in this case would have to cross 'r' links, $r \le 2n+1$, to meet its peer over dimension 2n. This is a total overhead of 2n+1 extra routing steps in the complete emulation, which may not be very large, for most applications. We call this a type–2 emulation.

Note that the routing overhead of the last step in a type–2 emulation can be reduced by taking one of the two sets, say S_1, and renumbering each node in each n–cube from (1,j,i) to $(1,j,\bar{i})$. In that case node (0,i,i) would be connected to $(1,i,\bar{i})$. Each node (1,j,i) in S_1 would then be at a distance of n+1 from (0,j,i), i.e. $(1,j,i) \rightarrow x \rightarrow (1,j,j) \rightarrow 1 \rightarrow (0,j,\bar{j}) \rightarrow n-x \rightarrow (0,j,i)$. The overhead is also reduced to n+1 in an ECC on account of the lower diameter of the ECC, as we shall see later. The routing overhead can be eliminated by connecting two MCCs together, to obtain what is called a Double Connection Cube or DCC [13].

The routing complexity of emulations is as follows: In a type–1 emulation there is the overhead of one exchange step for the total routing involved in emulating 2n+1 dimensions. Since two conflict free routing steps account for the exchange, we have a relative routing ratio of (2n+3) : (2n+1) between the MCC and the hypercube, which does not change the routing complexity. Even in a type–2 emulation there is no increase in the routing complexity over the hypercube. This is because the overhead is at most n+1 routing steps for every 2n+1 steps.

7: Scalability

An MCC of dimension 2n+1 can be expanded to an MCC of dimension 2n+3 by (1) Expanding each n–cube, in each of the two sets, into an (n+1)–cube and (2) Doubling the number of cubes in each set and making the requisite additional connections. *No changes to existing connections are necessary.* The process is as follows :

1) Convert n–cubes in each set into (n+1)–cubes by adding nodes (k,j,i) $\forall$ i,j,k $\ni 0 \le j < 2^n, 2^n \le i < 2^{n+1}$ and k = 0,1 and connecting (k,j,i) to $(k,j,i+2^n)$ $\forall$ i,j $\ni 0 \le i,j < 2^n$
2) Add 2^n more (n+1)–cubes $C_r, C_{r+1}, \ldots, C_{s-1}$ to each set, where $r = 2^n$ and $s = 2^{n+1}$. Now each set contains 2^{n+1} hypercubes of dimension n+1. Make the remaining connections as follows :
3) (0,j,i) to (1,i,j) $\forall$ i,j $\ni 0 \le j < 2^n$ and $2^n \le i < 2^{n+1}$
4) (0,j,i) to (1,i,j) $\forall$ i,j $\ni 2^n \le j < 2^{n+1}$ and $0 \le i < 2^{n+1}$.

This process quadruples the number of nodes while increasing the node degree by one.

8: The Enhanced Connection Cube or ECC

Adding a *single* extra link to each existing node of an MCC, converts it into a network which not only has the symmetry and the hypercube emulation capability of the MCC, but also has a diameter that is about *half* that of a comparable hypercube. This is the Enhanced Connection

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

Cube or the ECC. An MCC of dimension 2n+1 is converted into an ECC of dimension 2n+1 by adding the following connections between nodes in the same set:

Connect node (k,j,i) to node $(k,\bar{j},\bar{i})$ $\forall$ i, j $\ni 0 \le i,j < 2^n$ and k = 0,1.

This is a connection from a node in one n–cube to its complementary node in its complementary n–cube in the same set. We call these the **diagonal links**. A node in an ECC, therefore, has n internal links, one diagonal link and one interset link. This is a node degree of n+2 in a network of 2^{2n+1} nodes. Incidentally, as can be seen from figure 3, a

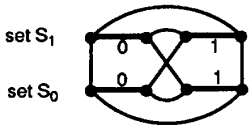

Fig 3. ECC of dimension 3

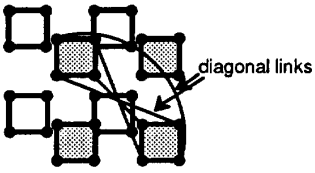

Fig 4. Some diagonal links in one set of an ECC of dimension 5

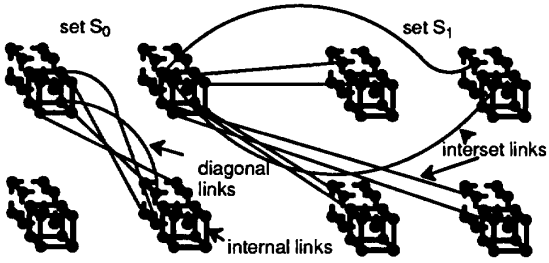

Fig 5. Some links in an ECC of dimension 7

3–dimensional ECC, constructed from two sets containing 1–dimensional hypercubes, has the same structure as a 3–dimensional twisted cube [7], a (2,1) Mcube [12] and a 3–dimensional Mobius cube [4]. It is interesting to note that all these structures, including the ECC, may be viewed as generalizations of the twisted 3–cube [11].

9: Routing and diameter for the ECC

In this Section we will develop a fully distributed routing algorithm for the ECC and will show that an ECC of dimension 2n+1, containing 2^{2n+1} nodes, has a diameter of n+2 under this routing algorithm, for n > 1, and a diameter of n+1 for n = 1. This translates to a network of N nodes having a node degree of $^1/_2(\log N) + {}^3/_2$, and a diameter that is also $^1/_2(\log N) + {}^3/_2$, in spite of this low node degree.

It is easy to verify from figure 3 that the diameter of an ECC of $2^{2(1)+1}$ nodes is 2 = 1+1. We will show, in the following theorem, that the diameter is n+2 for an ECC of 2^{2n+1} nodes for n > 1.

Theorem: $\exists$ a routing algorithm under which the diameter of an ECC of 2^{2n+1} nodes is n+2.

Proof: Note that each set S_0 and S_1, in the ECC consists of n–cubes of diameter n. Let **source = (k,j,i)** and **destination = (r,q,p)** where j, i, q and p are represented by n bits each, and k and r are represented by one bit each. The following routing can be used:

Case(I): r = $\bar{k}$ (either of two routes can be used)

Route1 : $(k,j,i) \to x_1 \to (k,j,q) \to 1 \to (r,q,j) \to y_1 \to (r,q,p)$

Route2 : $(k,j,i) \to x_2 \to (k,j,\bar{q}) \to 1 \to (r,\bar{q},j) \to 1 \to (r,q,\bar{j}) \to y_2 \to (r,q,p)$ $\{ 0 \le x_1, y_1, x_2, y_2 \le n \}$

Case(II): r = k

Route3 : $(k,j,i) \to x_3 \to (k,j,p) = (r,q,p)$ {if q = j}

Route4 : $(k,j,i) \to 1 \to (k,\bar{j},\bar{i}) \to x_4 \to (k,\bar{j},p) = (r,q,p)$ { if $q = \bar{j}$ }

Route5 : $(k,j,i) \to 1 \to (\bar{k},i,j) \to$ then follow Case(I) routing { if $q \ne j$ and $q \ne \bar{j}$ } $\{ 0 \le x_3, x_4 \le n \}$

Noting, as before, that the distance between nodes (k,j,i) and (k,j,q) is d(i,q) = the distance between nodes i and q in an n–cube using standard hypercube routing, we have, *length of Route1* = d(i,q) + d(j,p) + 1 and *length of Route2* = $d(i,\bar{q}) + d(\bar{j},p) + 2$.

Remember that if x and y are nodes in an n–cube then $d(x,y) = n - d(\bar{x},y) = n - d(x,\bar{y})$.

We will now show that

(length of Route1 > n+1) $\Rightarrow$ (length of Route2 $\le$ n+1).

Let length of Route1 > n+1

$\therefore$ d(i,q) + d(j,p) +1 > n+1

$\therefore$ n – d(i,q) + n – d(j,p) – 1 < n–1

$\therefore$ $d(i,\bar{q}) + d(\bar{j},p) + 2 < n+2$ $\therefore$ length of Route2 $\le$ n+1

$\therefore$ length of Route1 > n+1 $\Rightarrow$ length of Route2 $\le$ n+1.

Therefore every destination in Case(I) can be reached by a route of length at most n+1. It follows that Route5, in Case(II) has a maximum length of n+2. Route3 and Route4 have maximum lengths of n and n+1 respectively. $\therefore$ ***The diameter of the ECC is n + 2 under the above routing algorithm.***

Note that in an n–cube, $d(x,y) = |x \oplus y|$ = the number of set bits in $x \oplus y$. Hence the routing algorithm is:

```
if r = k̄ then
        if |i ⊕ q| + |j ⊕ p| ≤ n then use Route1;
        else use Route2;
else /* if r = k */
        if q = j use Route3 (hypercube routing);
        else /* if q ≠ j */
```


Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

if $q = \bar{j}$ then use Route 4;
else use Route5:

The algorithm can more elaborately be written in terms of target link types, as follows:

```
if r = k̄ then
   if i = q or (i = q̄ and |j ⊕ p| > 0) then
      send msg. to (r,i,j) over interset link;
   else
      if |i ⊕ q| + |j ⊕ p| ≤ n then
         send msg. to (k,j,q) over internal link;
      else send msg. to (k,j,q̄) over internal link;
else /* if r = k */
   if q = j then send msg. to (k,j,p) over internal link;
   else
      if q = j̄ then send msg. to (k,j̄,i) over diagonal link;
      else send msg. to (k̄,i,j) over interset link;
```

Figure 6 compares the diameter of the ECC with those of the hypercube and the HCN. Note that the ECC and the HCN are defined for network sizes that are odd and even powers of 2 respectively.

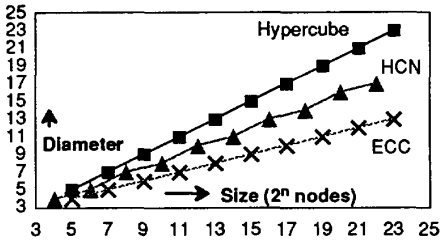

Fig 6. Comparative Diameters

# of nodes	hypercube	HCN	ECC
2^5	2.5	—	2.677
2^6	3.0	3.22	—
2^9	4.50	—	4.65
2^{10}	5.0	5.24	—
2^{13}	6.5	—	6.50
2^{14}	7.0	7.16	—
2^{17}	8.5	—	8.32
2^{18}	9.0	9.05	—
2^{21}	10.5	—	10.14
2^{22}	11.0	10.95	—
2^{23}	11.5	—	11.06

Table 1. Average Internode Distances

10: Uniform distance distribution (ECC)

The uniform distance distribution of the ECC follows from similar reasoning to that used in Section 4 for the MCC. The same is not true of the HCN [6] and the Hypernet [8] which have uneven distance distributions. Further, in spite of its low node degree, the average internode distance (average distance traveled by a message in the system) of the ECC is close to that of the hypercube (and, in fact, *lower* than that of the hypercube for very large networks). Table 1 shows the average internode distances of ECCs as compared to hypercubes and HCNs. The internode distances in the ECC (and the MCC) were calculated by simulating the distributed routing algorithm, explained above, on networks of various sizes. The average distance traveled by a message

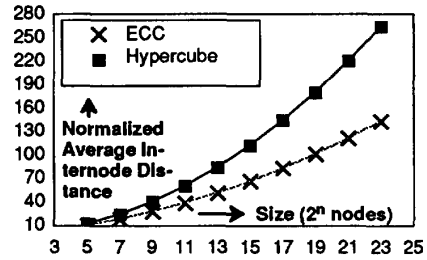

Fig 7. Normalized Average Distances

is a popular criterion in evaluating loosely coupled systems. However, after considering the physical limitations on the number of pins on a chip and the amount of power available to drive communication lines, it has been proposed [5] that constant bandwidth be assumed in single chip computers. Therefore the bandwidth available per port is inversely proportional to the number of ports per node. This leads to a more realistic 'normalized' average distance [2] which is the average distance multiplied by the number of ports per node. Figure 7 gives the comparative normalized average distances for the ECC and the hypercube.

Simulation results (obtained for MCCs and ECCs) indicate that if each node sends messages that are targeted at all other nodes with equal probability (uniform locality), *the rate of flow of messages through every node is the same, in each network. Further, it was confirmed that the distance distribution is uniform, i.e $D_j(i)$ is the same for all nodes j, in each network. This means that the Average Path Delay (APD) is the same for all nodes in each network.* The calculated values of the diameters were also confirmed. The simulation also showed that under uniform traffic, all links in the same category, (internal, diagonal or interset) are used with the same frequency. There is a difference in the usages of links in different categories. However, of the three categories, the interset links are used the most. This is not likely to be a serious drawback because in most practical implementations, there is likely to be a greater locality of traffic which might use the internal links in each n–cube, the most. The simulation was carried out using the fully distributed routing algorithms explained earlier.

11: Algorithm mapping on MCCs and ECCs

Let us consider a couple of common types of parallel algorithms and see how they can be executed on the MCC, (and therefore, on the ECC) with conflict–free routing and with the same efficiency as on a hypercube.

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

Tree–based Algorithms: In tree–based algorithms, the data paths emulate the connections of a tree structure. 2^r data elements are operated upon, in pairs, at the rth level of a hypothetical tree. One of the values from each pair is passed on to the next stage. This process continues until the root of the tree is reached and a single value is obtained. This is also called "recursive doubling". An example of such an algorithm is the parallel computation of the maximum value from a list of 'N' numbers. Let us assume that each node in the MCC contains $N/2^{2n+1}$ members of the list. The following algorithm can then be used to calculate the maximum.

1. Calculate local maximum at each node.
2. Send all local maximums from (0,j,i) to (1,i,j) $\forall$ i,j over interset links.
3. Calculate maximum in each n–cube into local node 0.
4. Send values from (1,q,0) to (0,0,q) $\forall$ q.
5. Calculate maximum in n–cube C_0 of set S_0.

Analysis: Step 1 requires $O(N/2^{2n+1})$ time.
Steps 3 thru 5 require O(2n+1) time. If $N = 2^{2n+1}$, the algorithm requires O(2n+1) time. This is the same as that required in a hypercube. In fact the number of steps is almost exactly the same as that required for a similar algorithm on a hypercube. The only additional steps are step 2 and step 4.

Ascend / Descend Algorithms: In ascend/descend algorithms, all data elements that are separated by 2^k positions, in a list, need to be operated upon in the ith step of the computation, where k is a function of i. A good example of such an algorithm is the Bitonic Merge Sort. Assume, for simplicity's sake, that each of the 2^{2n+1} nodes of the MCC contains one element 'A', of the list to be merged. Also assume that if A(x) denotes the element of the list in the node with binary address x, then for all nodes in set S_0, $x < y \Rightarrow A(x) \leq A(y)$ and for all nodes in set S_1, $x < y \Rightarrow A(x) \geq A(y)$:

0. Exchange A((0,j,i)) and A((1,i,j)) $\forall$ i,j , over interset links { this step ensures that consecutive values in each list are in different n–cubes in the same set }
1. Send A((1,i,j)) to (0,i,j) $\forall$ i,j { requires at most n+1 hops, as discussed in the type–2 emulation }.
2. Do a comparison in all nodes in set S_0.
3. Send lower valued elements from (0,i,j) to (1,i,j) $\forall$ i,j { routing same as 1 }.
4. For r = n–1 downto 0 do step 5

5a. For all nodes with binary address 'b' $\ni r^{th}$ bit in b = 1 send A(b) along internal link # r
5b. Compare values at all recipient nodes and return lower value back along link # r

6. Repeat Exchange step (Step 0.)
7. Repeat step 4.

Analysis: Steps 2, 4, 5 and 7 are exactly the steps required to do a similar Bitonic Merge Sort on a (2n+1)–dimensional hypercube. The only additional steps are the n routing steps of 1 and 3 and the exchange steps of 0 and 6 resulting in the same overall execution complexity as in a hypercube.

12: Advantages of the MCC and the ECC

The CCs (MCC and ECC) have several advantages, not only over the hypercube (table 2), but also over other proposed networks that have a similar capability of performing hypercube based algorithms in time complexities comparable to the hypercube with fewer links than a hypercube, i.e. the HCN [6] and the Hypernet [8] (table 3).

#Node	#links/ node	Diam- eter	#links/ node	Diam- eter	#links/ node	Diam- eter
2^7	5	5	4	8	7	7
2^{11}	7	7	6	12	11	11
2^{15}	9	9	8	16	15	15

Table 2. Comparative Characteristics of example ECCs, MCCs and Hypercubes

Network	#Nodes	#links/node	Diameter
ECC	2^{2n+1}	n+2	n+2
MCC	2^{2n+1}	n+1	2n+2
Hypernet (n+1,2)	2^{2n+1}	n+2	2n+3
Torus	$2^n \times 2^{n+1}$	4	2^{n+1}
Hypercube	2^{2n+1}	2n+1	2n+1
Hypercube	2^{2n}	2n	2n
HCN (n,n)	2^{2n}	n+1	$n+\lfloor n/2 \rfloor+1$

Table 3. General Topological Comparison of the ECC and MCC with other networks

1) 2^{2n+1} nodes are connected in the MCC with a node degree of n+1 and in the ECC with a node degree of n+2. The hypercube requires a node degree of 2n+1 to connect 2^{2n+1} nodes.
2) Typical parallel algorithms can run on both the CCs with the same time complexity as on a hypercube.
3) In spite of the their low link complexity CCs of 2^{2n+1} nodes can emulate the connections in a hypercube of 2^{2n+1} nodes, with no increase in routing complexity.
4) The CCs have uniform distance distributions. Not so, the HCN or the Hypernet.
5) Simulation results show that in CCs, under uniform traffic, the rate of flow of messages through every node is the same. This prevents certain nodes from being bottlenecks. Neither the HCN nor the Hypernet posses these qualities.
6) Like a hypercube, the MCC is scalable without changing any existing connections. The HCN is not scalable without changing some of the existing connections.

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.

7) The diameter of an ECC of 2^{2n+1} nodes is n+2, about 1/2 that of a comparable hypercube. The diameter of the MCC is about the same as that of a hypercube. The diameter of an HCN of 2^{2n} nodes is $n+\lfloor n/2 \rfloor+1$, about 3/4 that of a comparable hypercube. The diameter of (n+1,2) Hypernets is 2n+3.

8) The average internode distance of an ECC is about the same as that of a comparable hypercube and is, in fact, smaller for very large network sizes.

Note: Both ECC and MCC constructions can be adapted to have 2^{2n} (instead of 2^{2n+1}) nodes as follows:

Let S_0 *and* S_1 *be two sets each containing* 2^{n-1} *hypercubes of dimension* n *(*2^{2n-1} *nodes in each set).*

Let $C_0, C_1, \ldots, C_{N-1}$ *be the n–cubes each set, where* $N = 2^{n-1}$.

Using the (k,j,i) *notation,*

Connect node (0,j mod 2^{n-1},i) to node (1,i mod 2^{n-1},j)

$\forall$ i, j $\ni 0 \le i,j < 2^n$ (MCC).

Connect node (k,j mod 2^{n-1},i) to node (k,$\bar{j}$ mod 2^{n-1},$\bar{i}$)

$\forall$ i, j $\ni 0 \le i,j < 2^n$ and k = 0,1 (Diagonal links for ECC).

13: Conclusion

This paper proposes new static interconnection networks called the Minimal Connection Cube or MCC and the Enhanced Connection Cube or ECC, respectively, for large scale, loosely coupled systems. The ECC is a regular and symmetric network that can connect 2^{2n+1} nodes in such a way that each node has degree n+2. In spite of this low node degree the ECC has a diameter of n+2, i.e. about half the diameter of a comparable hypercube, and can emulate all the connections of a hypercube of dimension 2n+1, with no increase in routing complexity. The low node degree is a very useful factor in the construction of large–scale systems. The MCC is a regular and symmetric subnetwork of the ECC that can connect 2^{2n+1} nodes in such a way that each node has degree n+1. The MCC has about the same diameter as a comparable hypercube and a very slightly higher average internode distance. It too, can emulate all the connections of a hypercube, in the same time complexity as the hypercube. Both, the ECC and the MCC have uniform distance distributions and appear to be viable alternatives for constructing massively parallel systems.

References

[1] S. Abraham and K. Padmanabhan, "An Analysis of the twisted Cube Topology," *Proc. 1989 International Conference on Parallel Processing*, Vol. 1, pp.116–120.L.

[2] D. P. Agrawal, V. K. Janakiraman and G.C. Pathak, "Evaluating the performance of multicomputer configurations," *IEEE Computer*, vol. 19, pp. 23–37, May 1986.

[3] N. Bhuyan and D. P. Agrawal, "Generalized Hypercube and Hyperbus structures for a Computer Network," *IEEE Trans. on computers*, vol. C–33, No. 1, April 1984, pp. 323–333.

[4] P. Cull and S. Larson, "The Mobius Cubes," *Proc. 6th Distributed Memory Computing Conference*, (April 1991),pp.699–702.

[5] A. M. Despain and D. A. Patterson, "X–Tree: A tree structured multiprocessor computer architecture," *in Proc. 5th Annu. Symp. Comput. Architecture*,. Aug. 1978. pp. 144–151.

[6] K. Ghose and K. R. Desai, "The Design and Evaluation of the Hierarchical Cubic Network," *Proc. 1990 International Conference on Parallel Processing*, Vol. 1, pp. 355–362.

[7] P. A. J. Hilbers, M. R. J. Koopman and J. L. A. van de Snepscheut. "The TwistedCube," *Parallel Architectures and Languages Europe, Lecture Notes in Computer Science* (Jun. 1987), pp. 152–159.

[8] K. Hwang and J. Ghosh, "Hypernet: A Communication Efficient Architecture for Constructing Massively Parallel Computers," *IEEE Trans. on Computers*, Vol. C–36, No. 12, Dec 1987, pp. 1450–1466.

[9] S. Lakshmivarahan and S. K. Dhall, "A New Hierarchy of Hypercube Interconnection Schemes for Parallel Computers," Journal of Supercomputing, 2, 1988, pp. 81–108.

[10] F. P. Preparata and J. Vuillemin, "The Cube–Connected Cycles: A Versatile Network for Parallel Computation," *Comm. ACM*, Vol. 24, No. 5, May 1981, pp. 300–309.

[11] C. L. Seitz, "Concurrent VLSI Architectures," *IEEE Trans on Computers*, Vol. C–33(1984)1247–1265.

[12] N. K. Singhvi, "The Mcube", Master's Thesis, Dept. of Computer Science, SUNY, Binghamton, Fall 1991.

[13] N. K. Singhvi, "Connection Cubes: Symmetric, Low Diameter Interconnection Networks with Low Node Degree," Dept. of Computer Science, SUNY, Binghamton, Technical Report no. CS–TR–92–11.

[14] L. D. Wittie, "Communication Structures for Large Networks of Microcomputers," *IEEE Trans on Computers*, Vol. C–30, No. 4, April 81, pp. 264–273.

Authorized licensed use limited to: Qatar National Library. Downloaded on January 24,2026 at 20:20:12 UTC from IEEE Xplore. Restrictions apply.